# Debugging

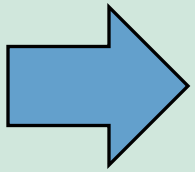
**COMP211 Connor McMahon**

# Announcements

- CLI checkoffs
  - List of questions will be released later today on Canvas (I'll post a link on the course schedule)
  - You'll meet with a TA during office hours who will ask you to perform a subset of the questions
  - You'll have two tries
  - Dates
    - Wednesday, Jan 22 through Wednesday, Feb 5
    - Early completion will earn extra credit.

# Arrays

How do we increment the count of the current character?



```c
#include <stdio.h>

#define NUM_DIGITS 10

int main() {
        int c;
        int ndigit[NUM_DIGITS] = {0};

        while ((c = getchar()) != EOF) {
                if (c >= '0' && c <= '9') {
                        // TODO
        }
}
```

A. ndigit[c]++;
B. ndigit['c' - '0']++;
C. ndigit[c - '0']++;
D. ndigit[c - 48]++;

ndigit is an array where each index represents a digit (0-9), and the value at that index represents the count of occurrences of that digit.

# Arrays

How do we increment the count of the current character?

```
#include <stdio.h>

#define NUM_DIGITS 10

int main() {
        int c;
        int ndigit[NUM_DIGITS] = {0};

        while ((c = getchar()) != EOF) {
                if (c >= '0' && c <= '9') {
                        ++ndigit[c - '0’];
                }
        }
}
```

# Arrays

- ndigit[c]++;
  - This directly uses the character c as the array index. Since c represents the ASCII value of the input character, the index will be between 48 and 57 for digits ('0' to '9'). ndigit only has 10 elements (indices 0-9), so this will index out of bounds.
- ndigit['c' - '0']++;
  - The expression 'c' - '0' is incorrect because 'c' is treated as a literal character, not the value of the variable c. 'c' - '0' = 99 – 48 = 51
- ndigit[c - '0']++;
  - This is the correct approach. Subtracting '0' from the character c converts the ASCII value of the digit (e.g., 48 for '0', 49 for '1') to its numeric equivalent (e.g., 0 for '0', 1 for '1').
- ndigit[c - 48]++;
  - Subtracting 48 (the ASCII value of '0') from c is functionally equivalent to subtracting '0'. This will correctly map the ASCII value of the digit to its numeric equivalent.
  - While this works, it is less readable. Using '0' instead of 48 makes the intent clear.

# Debugging an Input-Handling Program in C Using VSCode

- We'll be walking through how to debug the following program in VSCode.

```
#include <stdio.h>
int main() {
    int my_char;
    while ((my_char = getchar()) != EOF) {
        if (my_char >= 'a' && my_char <= 'z') {
            my_char = my_char - ('a' - 'A');
        }
        putchar(my_char);
        fflush(stdout);
    }
}
```

# Standard Output and Standard Input

- In C, **stdout** and **stdin** are two of the standard I/O streams that handle input and output for your program. They allow communication between your program and the outside world, such as the terminal or a file.
- stdout (Standard Output):
  - Purpose: It is used to display output from your program.
  - Default Destination: The terminal (console) where your program runs.
- stdin (Standard Input):
  - Purpose: It is used to receive input into your program.
  - Default Source: The terminal (keyboard input) where your program runs.

# Input Buffering

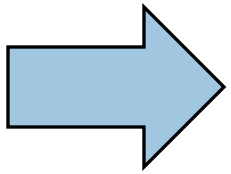
- Input from stdin is stored in a buffer in memory before it is passed to your program.
- When you type data at the terminal, it is not immediately delivered to the program. Instead, it waits in the buffer until:
  - The user presses **Enter**.
  - The buffer is full

# Output Buffering

- The text you send (e.g., using printf) is temporarily stored in a **buffer**.
- The buffer is flushed (output is sent to the terminal) when
  - A newline character (\n) is encountered.
  - The buffer is full.
  - The program explicitly flushes it using fflush(stdout).

# Debugging an Input-Handling Program in C Using VSCode

- Place a breakpoint in your program by clicking to the left of the line number (this area is known as the **gutter**) where you would like to place the breakpoint. A red dot should appear.



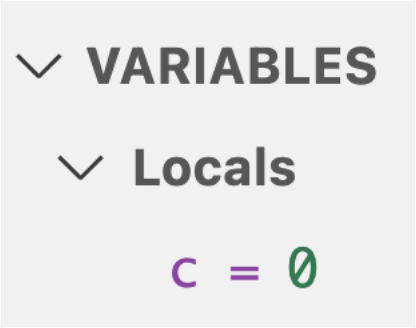
```
1   #include <stdio.h>
2   int main() {
3       int my_char;
4       while ((my_char = getchar()) != EOF) {
5           if (my_char >= 'a' && my_char <= 'z') {
6               my_char = my_char - ('a' - 'A');
7           }
8           putchar(my_char);
9           fflush(stdout);
10      }
11  }
```

# Debugging an Input-Handling Program in C Using VSCode

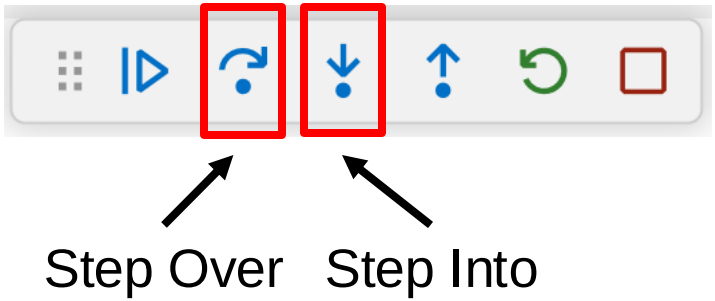
- In the upper right-hand corner, click the debug button.

- This panel should appear. You'll use this to control stepping through the program.

- Notice on the left panel that you can see the value of the local variable.



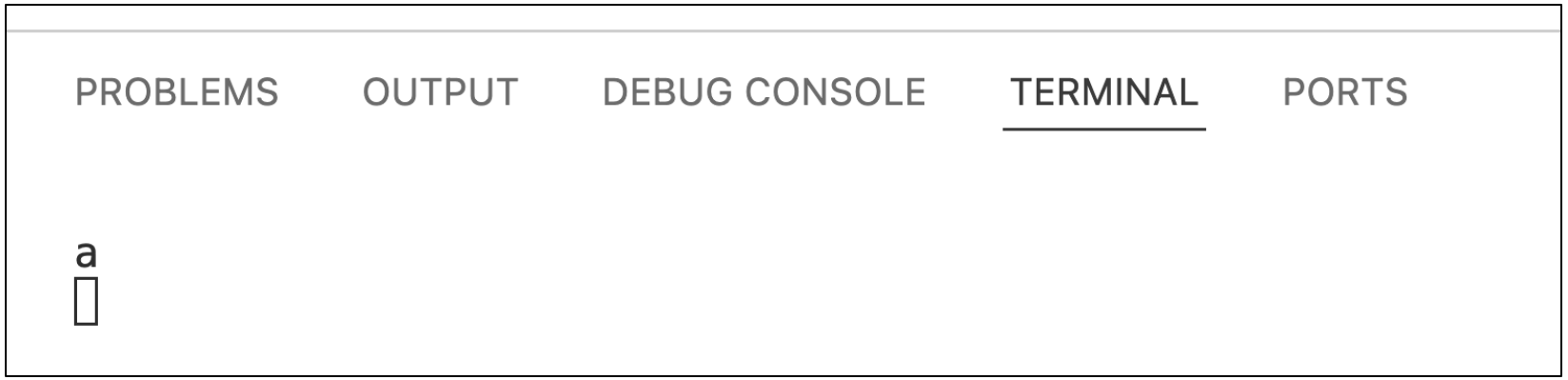
# Step Over vs Step Into




| | Step Over | Step Into |
|---|---|---|
| Definition | Executes the current line of code completely but skips stepping into any functions that are called on that line. | Executes the current line and pauses inside any function that is called, allowing you to debug the function line by line |
| Use Case | When you are not interested in the internal details of a function and just want to proceed to the next line of the current function. | When you want to examine the internal workings of a function being called. |
| What Happens | If the line contains a function call, the function will execute entirely instead of stepping through the internal lines of the function. | The debugger pauses at the first line of the function being called. You can then continue to step through the function's execution line by line. |

# Debugging an Input-Handling Program in C Using VSCode

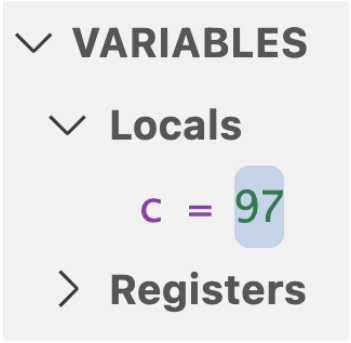
- We don’t have any need to step into getchar(), so we will click step-over.
- Notice that the program does not proceed. This is because getchar() is waiting to read input. In the terminal, type the letter `a` and click enter.




- Your program should have now moved onto line 5 of main.

# Debugging an Input-Handling Program in C Using VSCode

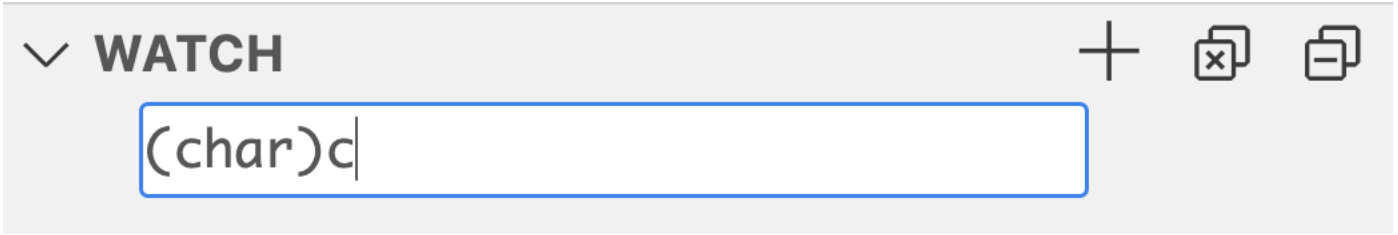
- If you examine the variables panel, you'll notice that the value of c has been updated to 97. Since c is of type int, it is displayed in decimal format.



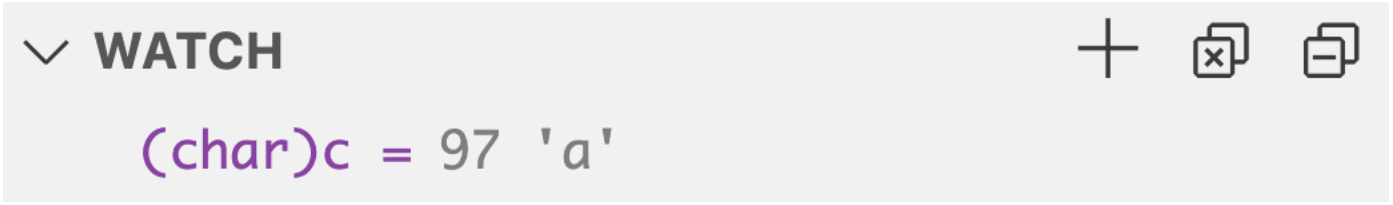

- Unless you have all of the ASCII values memorized, this value is not very useful to us. On the next slide, we'll see how to view the char value of c

# Debugging an Input-Handling Program in C Using VSCode

- To view the ASCII value of c, go to the Watch panel (located just below the Variables panel) and click the plus icon. In the input box, cast c to the char type and press Enter.
  - A watchpoint allows you to monitor a specific variable or memory location during program execution.




- You should now see the char value of c



# Debugging an Input-Handling Program in C Using VSCode

- Since we entered a lowercase letter, we can expect this if condition to evaluate to true.
- If we step again, the debugger will move into the if statement.

```
1   #include <stdio.h>
2   int main() {
3       int my_char;
4       while ((my_char = getchar()) != EOF) {
5           if (my_char >= 'a' && my_char <= 'z') {
6               my_char = my_char - ('a' - 'A');
7           }
8           putchar(my_char);
9           fflush(stdout);
10      }
11  }
```

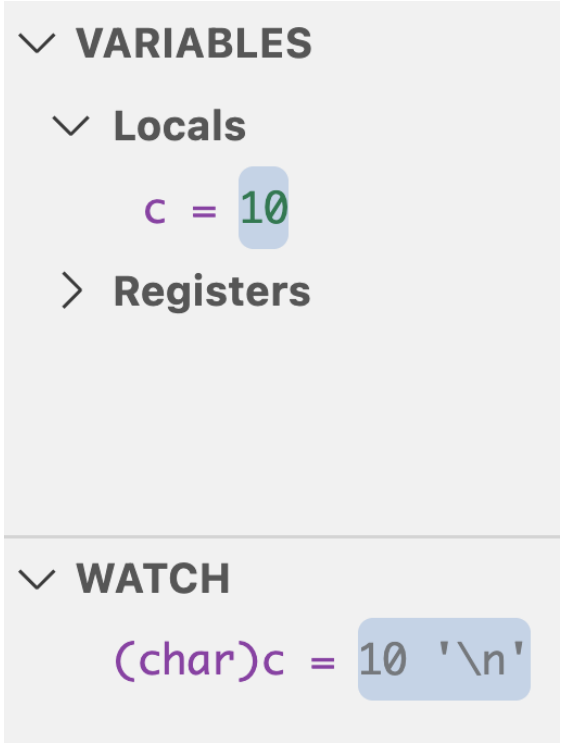
# Debugging an Input-Handling Program in C Using VSCode

- Step again (step over and step into will be the same in this case). The variable c is now storing an upper-case A.


- Step over putchar() and fflush(stdout). A capital A should have been printed to your terminal.

# Debugging an Input-Handling Program in C Using VSCode

- On this next iteration, the loop will be processing the newline character that you entered earlier.




- Step all the way through this iteration of the loop and see the newline character get printed to your terminal.

# Debugging an Input-Handling Program in C Using VSCode

- When you get back to the top, enter a longer string (e.g. “HeLlo”).
- Step through the program with this input to get more practice.

# Debugging an Input-Handling Program in C Using VSCode

- There's a bug in the following program! We'll step through the first few lines together, then it will be your job to find and resolve the bug!

```c
#include <stdio.h>

#define NUM_DIGITS 10

int main() {
        int c, i, nwhite, nother;
        int ndigit[NUM_DIGITS] = {0};
        nwhite = nother = 0;

        while (c = getchar() != EOF) {
                if (c >= '0' && c <= '9') {
                        ++ndigit[c - '0'];
                } else if (c == ' ' || c == '\n' || c == '\t') {
                        ++nwhite;
                } else {
                        ++nother;
                }
        }

        for (i = 0; i < NUM_DIGITS; ++i) {
                printf("%d: %d\n", i, ndigit[i]);
        }
        printf("white space: %d\nother: %d\n", nwhite, nother);
}
```

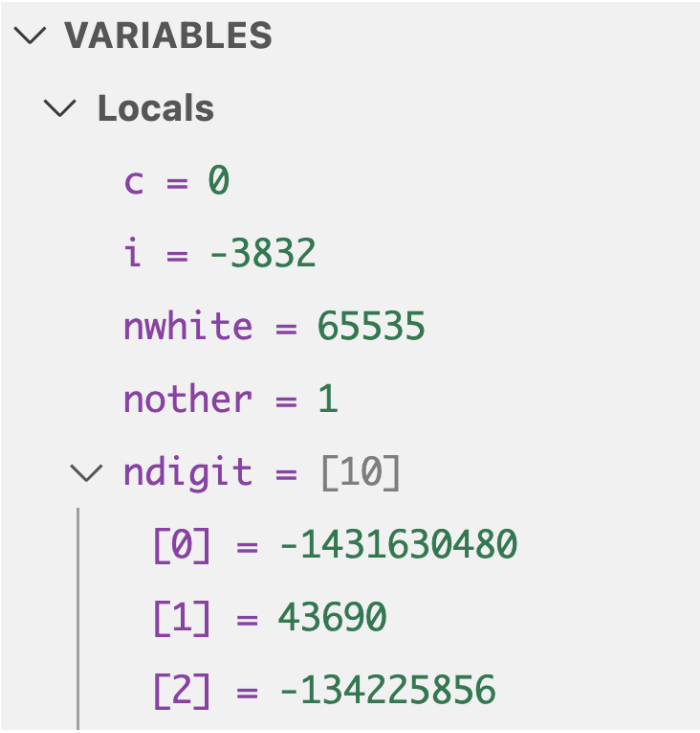
# Debugging an Input-Handling Program in C Using VSCode

- This program counts and categorizes characters from input as follows:
  - **Digits (0-9):** Tracks the frequency of each digit in an array ndigit.
  - **Whitespace:** Counts spaces, tabs, and newlines as nwhite.
  - **Other Characters:** Counts all characters that are neither digits nor whitespace as nother.

```c
#include <stdio.h>

#define NUM_DIGITS 10

int main() {
        int c, i, nwhite, nother;
        int ndigit[NUM_DIGITS] = {0};
        nwhite = nother = 0;

        while (c = getchar() != EOF) {
                if (c >= '0' && c <= '9') {
                        ++ndigit[c - '0'];
                } else if (c == ' ' || c == '\n' || c == '\t') {
                        ++nwhite;
                } else {
                        ++nother;
                }
        }

        for (i = 0; i < NUM_DIGITS; ++i) {
                printf("%d: %d\n", i, ndigit[i]);
        }
        printf("white space: %d\nother: %d\n", nwhite, nother);
}
```

# Debugging an Input-Handling Program in C Using VSCode

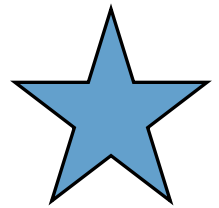
- Start by placing your breakpoint on line 7. Then, press the debug button.
- Note that all the variables contain "garbage" values, as they have not yet been initialized. Note that your values may differ from those shown in the example below.
- A **garbage value** is a random, unpredictable value that exists in a variable because it hasn’t been initialized.



# Debugging an Input-Handling Program in C Using VSCode

- Step to line 8. You should see that every element in ndigit is now initialized 0.
- Step again and notice that nwhite and nother are also now initialized to 0.
- Now, it’s your turn to take over and find the bug!

# Correct Implementation



```
#include <stdio.h>

#define NUM_DIGITS 10

int main() {
        int c, i, nwhite, nother;
        int ndigit[NUM_DIGITS] = {0};
        nwhite = nother = 0;

        while ((c = getchar()) != EOF) {
                if (c >= '0' && c <= '9') {
                        ++ndigit[c - '0'];
                } else if (c == ' ' || c == '\n' || c == '\t') {
                        ++nwhite;
                } else {
                        ++nother;
                }
        }

        for (i = 0; i < NUM_DIGITS; ++i) {
                printf("%d: %d\n", i, ndigit[i]);
        }
        printf("white space: %d\nother: %d\n", nwhite, nother);
}
```